

Memory Management**6.1 : Memory Management Concepts**

Q.1 List the Memory Management Function.

Ans. : Functions are list below :

1. Allocate primary memory space to processes.
2. Minimize access time.
3. Determining allocation policy for memory.
4. Deallocation technique and policy.

Q.2 Explain concept of dynamic relocation.

Ans. : • Base register is sometimes called as a relocation register. The value of the relocation register is added to every address generated by a user process at the time it is sent to main memory.

- User can load a process with only absolute addresses for instructions and data, only when those specific addresses are free in main memory. Program's instruction, data and any other data structure required by the process can be accessed easily if the addresses are relative.

- Fig. Q.2.1 shows dynamic relocation.

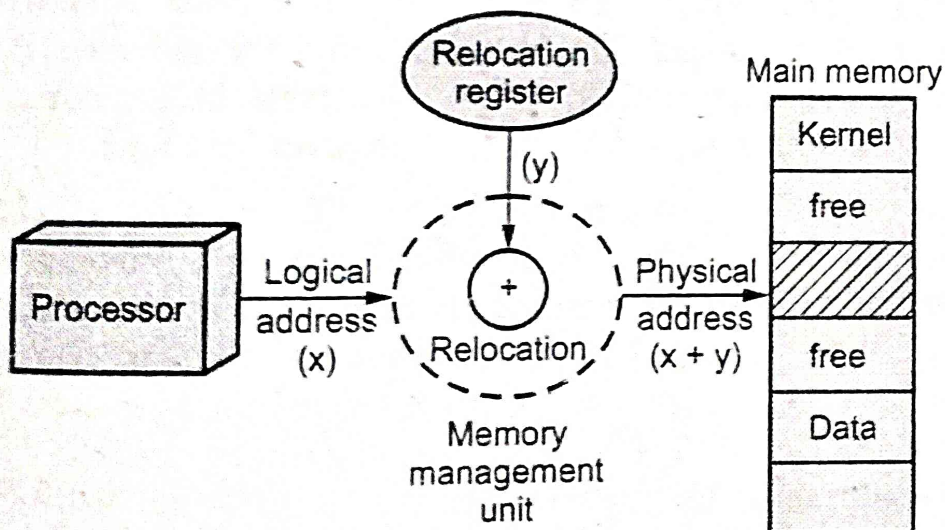


Fig. Q.2.1 Dynamic relocation

- User programs never reads the main memory physical address. Dynamic relocation requires extra hardware. It is mapping of the virtual address space to the physical address space at run time.
 - Dynamic relocation makes it possible to move a partially executed process from one area of main memory into another without affecting other process.
 - Problem with relocation is that, it is necessary to perform an addition and a comparison on every memory reference. For good memory management, logical address space is bound with a separate physical address space.
- Q.3 How the user program is processed ? Also explain binding of data and instruction to memory.**

Ans. : Address Space Mapping

- Secondary storage device stores program in binary executable format. Before executing, the program is loaded into the main memory.
- Most of the operating systems allow a user process to store in any section of the main memory. Source program uses symbolic addresses.
- Binding of instruction and data to main memory address is following ways : 1. Compile time 2. Load time 3. Execution time
- **Compile time** : Source program is translated at compile time to produce a relocatable object module. At compile time, the translator generates code to allocate storage for the variable. This storage address is used for code reference. Target address is unknown at compile time, it cannot be bound at compile time. Example of compile time binding is MS DOS.com programs.
- **Load time** : Compiler generates relocatable code if compile time binding is not performed. The loader modifies the addresses in the load module at load time to produce the executable image stored in main memory. Final binding is delayed until program load time.
- **Execution time** : Memory address of the program is changed at execution time, then execution time binding is used. Binding is delayed until the run time of the program. Normally all operating system uses execution time binding. Special hardware is used for execution time binding.

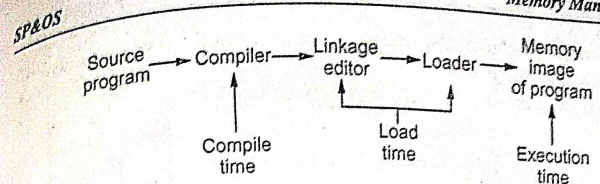


Fig. Q.3.1 Processing of user program

- Memory allocation and deallocation is done using run-time support of the programming language in which a program is coded. Allocation and deallocation requests are made by calling appropriate routines of the run time library.
- Kernel is not involved in this kind of memory management.

6.2 : Memory Partitioning

Q.4 Explain contiguous and non-contiguous memory allocation policies with suitable example. [SPPU : May-18, End Sem, Marks 8]

Ans. : 1. Contiguous memory allocation :

- In a fixed size partitioning of the main memory all partitions are of the same size. The operating system divides main memory into a number of fixed size partition. Each partition holds a single program.
- Fixed sized partitions are relatively simple to implement. Degree of multiprogramming depends on the number of partitions.
- Normally main memory is divided into two partitions :
1. For resident program 2. For user processes
- Batch operating system uses the fixed size partition scheme. The operating system keeps the record of memory allocation and deallocation in the form of table. Initially all the memory is available for user processes.
- When the process arrives in the system and needs memory, operating system search for large hole for this process. Hole is one large block of free available memory. If any free hole is found, process is allocated to the free hole of memory as is needed.
- After allocating number of holes for the processes, a set of various size holes is scattered throughout memory at any given time. When a process arrives and searches for (memory) set of holes. But the holes must be large enough to accommodate the process.

- Fixed sized partitions are simple to implement. Any process whose size is less than or equal to the partition size can be loaded into any available partition.

- Fig. Q.4.1 shows an example of fixed partitioning of a memory.

- There are two difficulties with the use of equal size fixed partitions :

1. A program may be too big to fit into a partition.

2. Memory utilization is extremely inefficient.

- First difficulty is solved by using overlays. Overlays involves moving data and program segments in and out of memory.

2. Non-contiguous memory allocation :

- The use of unequal size partitions provides a degree of flexibility to fixed partitioning. In dynamic partitioning, the partitions are of variable length and number.

- In noncontiguous memory allocation, a program is divided into blocks that the system may place in nonadjacent slots in main memory.

- This allocation method do not suffer from internal fragmentation, because a process partition is exactly the size of the process.

- Fig. Q.4.2 shows noncontiguous memory allocation method. Operating system maintain the table which contains the memory areas allocated to process and free memory. Memory management unit use this information for allocating processes.

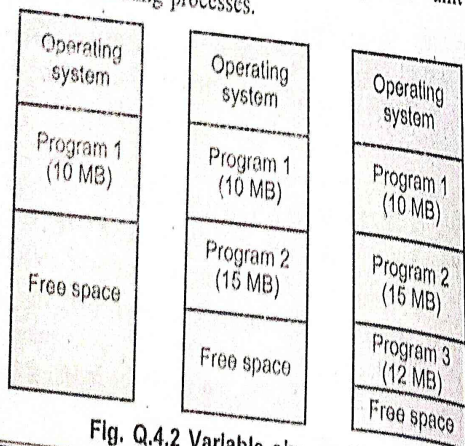


Fig. Q.4.2 Variable size partition

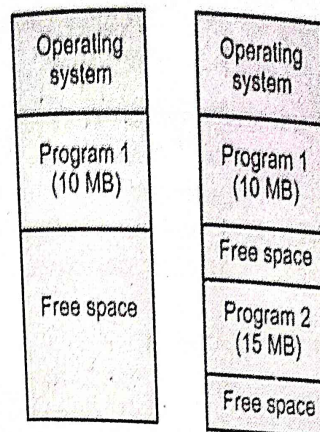


Fig. Q.4.3 Noncontiguous memory allocation

- Table contains information about memory starting address for process/program and their size.
- CPU sends the logical address of the process to the MMU and the MMU uses the memory allocation information stored in the table for calculating logical address. We called this address as effective memory address of the data/instruction.

Q.5 Differentiate internal and external fragmentation.

[SPPU : May-18, End Sem, Marks 4]

Ans. :

Internal Fragmentation	External Fragmentation
Internal fragmentation is the area occupied by a process but cannot be used by the process.	External fragmentation exists when total free memory is enough for the new process but it's not contiguous and can't satisfy the request.
First fit and best fit memory allocation does not suffer from internal fragmentation.	First fit and best fit memory allocation suffers from external fragmentation.
In fixed partitioning, inefficient use of memory due to internal fragmentation.	In dynamic partitioning, inefficient use of processor due to need for compaction to counter external fragmentation.

Paging's suffer from internal fragmentation.	Paging does not suffer from external fragmentation
Segmentation does not suffer from internal fragmentation.	Segmentation suffer from external fragmentation

Q.6 Compare contiguous and non-contiguous memory allocation.

[SPPU : Dec.-18, End Sem, Marks 3]

OR Compare fixed and variable sized partitioning.

[SPPU : May-19, End Sem, Marks 3]

Ans. :

Sr. No.	Contiguous allocation	Noncontiguous allocation
1.	Program execution take place without overhead.	Address translation is overhead.
2.	Swapped-in processes are placed in the original area.	Swapped-in processes can be placed anywhere in memory.
3.	Suffer from internal fragmentation.	Only paging, suffers from internal fragmentation.
4.	Allocates single area of memory for process.	Allocates more than one block of memory for process.
5.	Wastage of memory.	No wastage of memory.

Q.7 What is fragmentation ? Explain types of fragmentation with suitable diagram/example.

[SPPU : Dec.-18, End Sem, Marks 5, May-19, End Sem, Marks 3]

Ans. : • As processes are loaded and removed from memory, the free memory space is broken into little pieces. It happens after sometimes that processes cannot be allocated to memory blocks considering their small size and memory blocks remains unused. This problem is known as Fragmentation.

• Fragmentation are of two types :

1. Internal fragmentation.
2. External fragmentation.

• In fragmentation, OS cannot use certain area of available memory.

DECODE®

A Guide for Engineering Students

Internal fragmentation

- There is wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition is called as internal fragmentation.

• Fig. Q.7.1 shows an internal fragmentation.

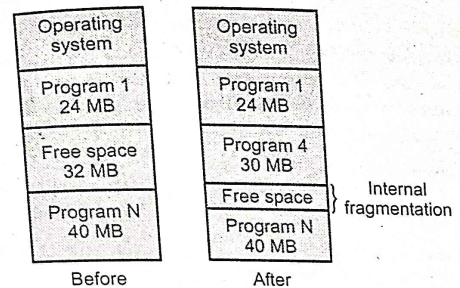


Fig. Q.7.1 Internal fragmentation

- To keep track of this free hole is overhead for system.

External fragmentation

- It occurs when enough total main memory space exists to satisfy a request, but it is not contiguous, storage is fragmented into a large number of small holes.

• Fig. Q.7.2 shows external fragmentation.

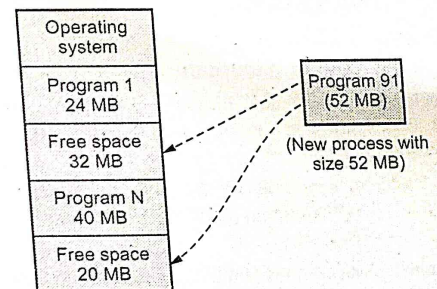


Fig. Q.7.2 External fragmentation

DECODE®

A Guide for Engineering Students

- Following are the solution for external fragmentation :

1. Compaction
2. Logical address space of a process/program

Q.8 How memory is protected by using relocation and limit register ?

Ans. : • Memory allocated to the process need to be protected against interference by other processes. Memory protection provides this facility. Relocation register and limit register are used for memory protection. Physical address is stored in the relocation register and range of virtual address is stored in the limit register.

- Fig. Q.8.1 shows logical address space.

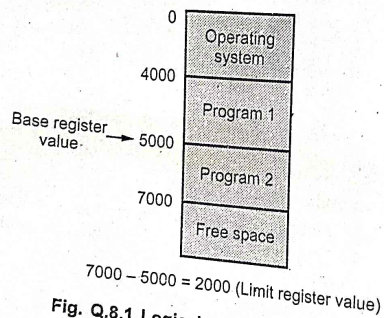


Fig. Q.8.1 Logical address space

- Fig. Q.8.2 shows memory protection by using relocation and limit registers. (See Fig. Q.8.2 on next page.)
- Use of registers method for memory protection requires proper arrangement. This method has to be used individually for each memory area allocated to a process.
- Every logical address generated by the CPU is checked against these registers. A protection violation interrupt is raised if any of these checks fail.
- Operating system load a base register and limit register. OS uses privileged instruction for this purpose. Operating system executes in monitor mode and privilege instruction also executes in privilege mode.

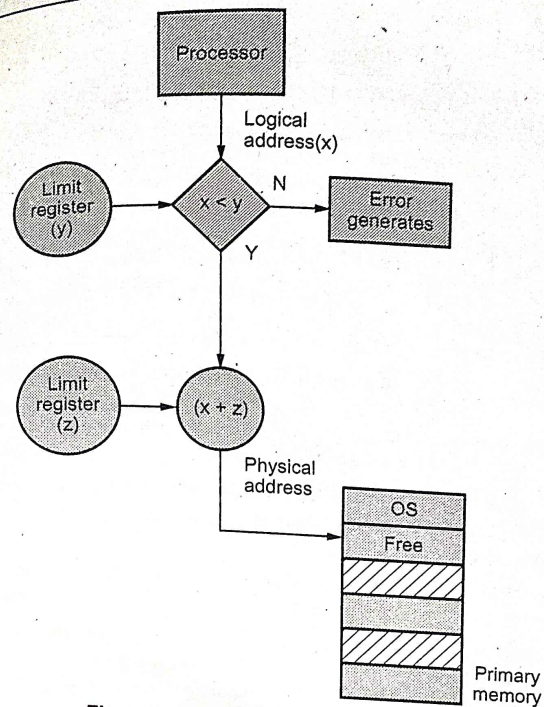


Fig. Q.8.2 Memory protection hardware

- Operating system prevents the user programs from changing the content of the registers. While executing in the monitor mode, operating system is given unrestricted access to both monitor and user's memory.
- In a multiprogramming environment, protection of main memory is essential. Paging or segmentation or both in combination provides an effective means of managing main memory.

Q.9 Explain the term in brief : Compaction.

[SPPU : May-18, End Sem, Marks 2]

Ans. : • Compaction solves problem of external fragmentation. Fig. Q.9.1 shows compaction.

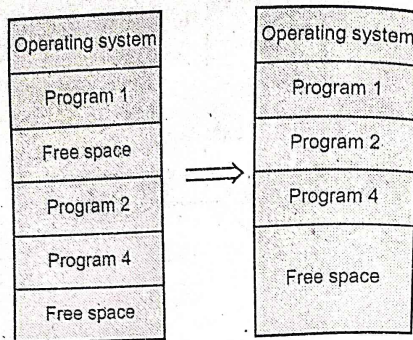


Fig. Q.9.1 Compaction

- Operating system moves all the free holes to one side of main memory and creates large block of free size.
- It must be performed on each new allocation of process to memory or completion of process for memory. System must also maintain relocation information.
- All free blocks are brought together as one large block of free space. Compaction requires dynamic relocation.
- Compaction has a cost and selection of an optimal compaction strategy is difficult. One method for compaction is swapping out those processes that are to be moved within the memory and swapping them into different memory locations.

6.3 : Buddy System

Q.10 Describe with example Buddy system. What are the advantages and disadvantages of Buddy system ?

Ans. : • The buddy system is a simple dynamic storage allocation method. The buddy system is a memory allocation and management algorithm that manages memory in power of two increments. Linux operating system manages memory using buddy algorithm.

- Fig Q.10.1 shows buddy system.

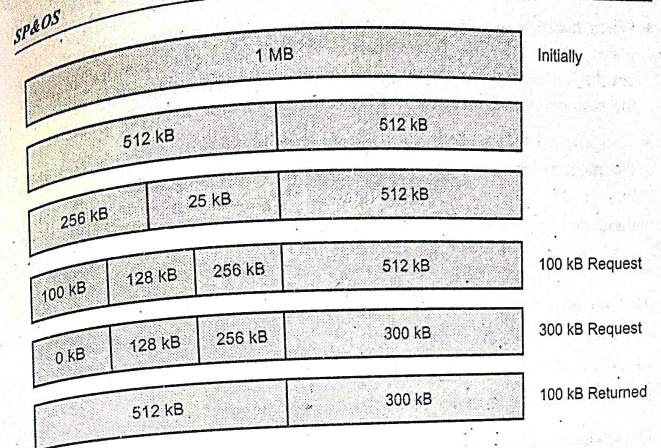


Fig. Q.10.1 Buddy system

- The buddy system assumes that memory is of size 2^N for some integer N . Both free and reserved blocks will always be of size 2^k for $k < N$. At any given time, there might be both free and reserved blocks of various sizes.
- The buddy system keeps a separate list for free blocks of each size. There can be at most N such lists, because there can only be N distinct block sizes.
- Single allocation block to be split, to form two blocks half the size of the parent block. These two blocks are known as 'buddies'.
- Initially memory consists of a single contiguous area of 128 pages. When user send request for memory, it is first rounded up to a power of 2.
- All the lists are initially empty, except for the 1 MB list, which has one hole listed. All allocations are rounded up to a power of two. Suppose the request for memory is 75 K then allocations rounded up to 128 K, 13 K allocations rounded up to 16 K, etc.
- If a block of that size is free, it is taken; otherwise, the smallest free block larger than the desired size is found and split in two halves. This splitting continues until the appropriate size is reached.

- When memory is deallocated, the buddy system groups contiguous free pages. When process free a block of memory, the memory manager checks the bitmap for checking block size. If adjacent block is also free, the memory manager combines the two blocks into a larger block.
- This method makes deallocation fast. If a block of size 2^k is free then the memory manager checks only the list of 2^k holes to merge them into a 2^{k+1} sized partition. Internal fragmentation is caused since memory requests are fitted in 2^k sized partitions.

Advantages :

- Easy to implement a buddy system
- Allocates block of correct size
- It is easy to merge adjacent holes
- Fast to allocate memory and de-allocating memory

Disadvantages :

- It requires all allocation unit to be powers of two
- It leads to internal fragmentation.
- Merging of holes is recursive, so poor worst-case behavior.

6.4 : Paging

Q.11 What is paging ? Explain address translation in paging.

Ans. : • In paging, operating system divides each incoming programs into pages of equal size. The sections of a disk are called block or sectors. The sections of main memory are called page frames. One sector will hold one page of job instructions and fit into one page frame of memory.

- In paging, logical address space of a program can be noncontiguous. It solves external fragmentation problem.
- The relation between virtual addresses and physical memory addresses given by page table.
- Fixed sized blocks are called frames and breaking of logical memory into blocks of same size called pages.

- Memory manager prepares following things before executing a program :

- Find out the number of pages in the program.
- Free space in the main memory.
- Loading of all the programs pages into memory.

- Pages are not loaded continuously in main memory. Each page can be stored in any available page frame anywhere in main memory. Memory manager keeps the track of pages of program. Paging avoids external fragmentation and need for compaction.

- Memory manager uses three tables to keep track of process and memory.

1. **Job table :** It stores the size of the active job and memory location where its page table is stored.

2. **Page map table :** It contains vital information about each page. PMT contains a page number and corresponding page frame memory address. It includes only one entry per page. Page numbers are in sequential order.

3. **Memory map table :** MMT is used to store page frame location and its status.

- Page size depends upon underlying hardware. Operating system maintains a page table for each process. The page table shows the frame location for each page of a process. Fig. Q.11.1 shows the virtual address format for paging system.

Page number (P)	Displacement or offset (D)
-----------------	----------------------------

Fig. Q.11.1 Virtual address format

- Processor hardware performs the logical to physical address translation. Logical address contains page number and offset. The physical address contains frame number and offset. Offset is a relative factor. It is used to locate that line within its page frame.

Q.12 What is difference between page and a frame ?

Ans. : • In paging, operating system divides each incoming programs into pages of equal size. The sections of a disk are called block or sectors. The sections of main memory are called page frames.

- Fixed sized blocks are called frames and breaking of logical memory into blocks of same size called pages.
- Pages are not loaded continuously in main memory. Each page can be stored in any available page frame anywhere in main memory.
- Page frames on main memory are required in paging. No frames are required in segmentation.

Q.13 Explain design issues in paging. Explain with example.

Ans. : Design issues for paging systems :

- Local versus Global allocation policies
 - Load control
 - Page size
 - Shared pages
 - Cleaning policy
- Local versus Global allocation policies
 1. Local replacement policy : Choose among resident set of process that generated page fault
 2. Global replacement policy : Choose among all pages eligible for replacement in main memory
 - 3. Types of allocations :
 - a. Fixed allocation, Local scope. Allocations cannot be too large or too small as it would be irreparable.
 - b. Variable allocation, Global scope
 - c. Difficult to obtain a good replacement policy.
 - d. Variable allocation, Local scope
 - e. Re-evaluate allocation from time to time
 - Load control : Trashing can be expected :
 - a. System will spend a lot of time copying data from disk to memory;
 - b. If the wrong pages are evicted from memory they will have to be loaded again very soon. This results in a lot of wasteful copying.
 - Page size : Determining best size requires balancing several competing factors. Choose a page size too big : Possibility for wasted memory. Choose a page size too small : OS needs to maintain larger page table.
 - Shared pages : Sometimes it is useful to share pages among processes. For example : User may be running several programs that use the same library; more efficient to share the pages, avoiding having multiples copies of the same page.

- Problem is that not all pages are sharable : Pages that are read-only, such as program text, can be shared; for data pages sharing is more complicated. When two or more processes share some code; special care needs to be employed with the shared pages.
- Cleaning policy : Paging works best when, there is an abundant supply of free page frames; such frames can be claimed as page faults occur. If every page frame is full and modified before a new page can be brought in, old page must first be written to disk.

Q.14 What elements are typically found in a page table entry ? Briefly define each element.

Ans. : • The region table entry contains a pointer to a table of physical page numbers called a page table.

- Page table entries may also contain machine dependent information such as permission bits to allow reading or writing of the page. The Kernel stores page tables in memory and accesses them like all other Kernel data structures.

- Fig. Q.14.1 shows Page table entry.

Page frame number	Age	Copy on write	Modify	Reference	Protect

Q.14.1

1. Page frame number is the real number in the main memory.
2. Age indicates total time period of frame without being referred in the memory.
3. Copy on write : Set when more than one process shares a page.
4. Modify field indicates page has been modified.
5. Reference : Page is referred or not.
6. Valid : Page is valid if it is in main memory.
7. Protect : Whether write operation is allowed or not.

Q.15 How TLB is used with paging?

Ans. : Paging with TLB : • Set of registers are used for implementing page table. Registers are suitable only for small page table. If page table size increases then special purpose hardware cache is used for page table.

- Special cache memory called a Translation Lookaside Buffer (TLB) is used with the address translation hardware. The full page table is kept in primary memory.
- TLB is very effective in improving the performance of page frame access. The cache buffer is implemented in a technology which is faster than the primary memory technology.
- Fig. Q.15.1 shows paging with TLB.

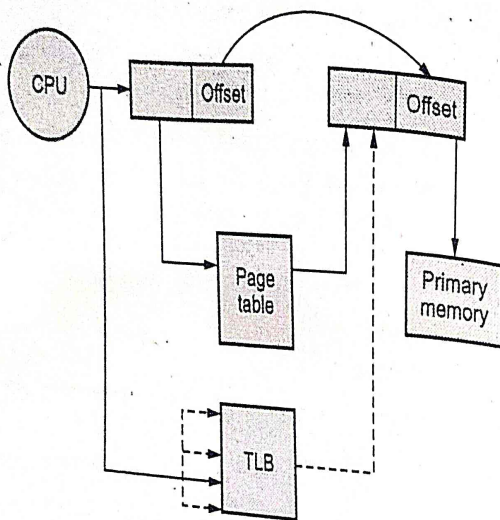


Fig. Q.15.1 Paging with TLB

- The TLB contents may be controlled by the operating system or by hardware, depending on the architecture.
- When a page is first translated to a page frame, the map is read from primary memory into the TLB. The TLB entry contains the page number, page frame's physical address etc.
- The page mapping mechanism first tries to find page number in the TLB. If the TLB contains the page number, then it is called as **page hit** or **TLB hit**.
- If the TLB does not contain an entry for page p, then it called as **page miss** or **TLB miss**. In case of TLB miss, the operating system locates the page table entry in the primary memory, which increases execution time.

- The TLB is associative, high-speed memory. A translation buffer is used to store a few of the translation table entries. It is very fast, but only for a small number of entries. On each memory reference.

1. First ask TLB if it knows about the page. If so, the reference proceeds fast.
2. If TLB has no information for page, must go through page and segment table to get information. Reference takes a long time, but gives the info for this page to TLB so it will know it for next reference.

- The greatest performance improvement is achieved when the associative memory is significantly faster than the normal page table lookup and the hit ratio is high. The hit ratio is the ratio between accesses that find a match in the associative memory and those that do not.

- **Disadvantage of TLB is that :** If two pages use the same entry of the memory, only one of them can be remembered at once. If process is referencing both pages at same time, TLB does not work very well.

- Each entry in the TLB must include the page number as well as the complete page table entry. The processor is equipped with hardware that allows it to simultaneously interrogate a number of TLB entries to determine if there is a match on page number. This technique is referred to as *associative mapping*.

Q.16 Explain working of hashed page table with diagram.

Ans. : • Hashed page table support address space of 32 bits and more. Hash value is used as virtual page number.

- Hash table contains a link list of elements. Each elements consists of following fields :

1. Virtual page number
2. Mapped page frame value.
3. Pointer to the next element.

- Hash table improves search speed.

- Fig. Q.16.1 shows block diagram of hashed page table.

Working

1. Virtual page number is taken from virtual address.
2. Virtual page number is hashed into the hash table.
3. Virtual page number is compared with the first element of linked list.

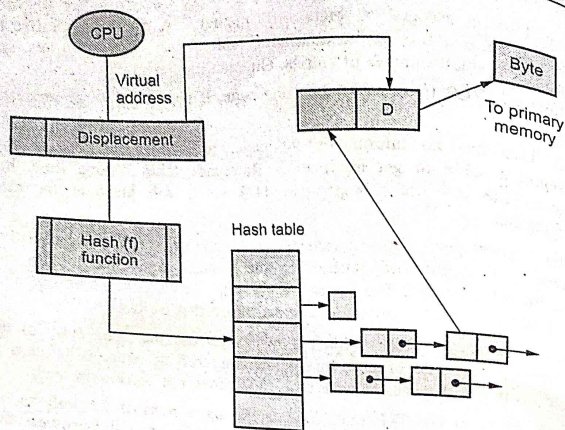


Fig. Q.16.1 Hashed page table

4. Both the value is matched, that value is (i.e. page frame) used for calculating physical address.
 5. If the both value is not matched, the entire linked list is searched for a matching.
- Clustered page table is same as hashed page table but only difference is that each entry in the hash table refers to several pages.

6.5 : Segmentation

Q.17 Explain segmentation using suitable example in brief.

[SPPU : Dec.-18, End Sem, Marks 3]

Ans. : Segmentation

- In segmentation, a program's data and instructions are divided into blocks called segments. A segment is a logical entity in a program.
- Logical view : A process consists of a set of segments.

- **Physical view** : It consists of non-adjacent areas of memory allocated to segments.
- All segment size may be equal or may not be equal. Segmentation support user view of memory. Fig. Q.17.1 shows programmer's view of a program.

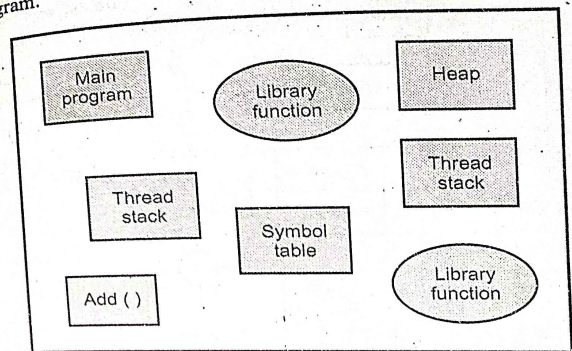


Fig. Q.17.1 Programmer's view

- Segmentation can be implemented with or without paging.
- Collection of segment is called as logical address space. Each segment is identified by its name.
- Processor generates logical addresses. These addresses consists of a segment number and an offset into the segment. Segment number is used as an index to segment page table.

Segment number	Offset
----------------	--------

Fig. Q.17.2 Logical address

- Segment names are normally symbolic names.
- Operating system maintains a segment table for each process. It is usually stored in main memory as a segment that is not to be loaded as long as the process can run.
- Each entry in the segment table has a segment base and a segment limit. The base field contains the segment relocation register for the target segment. The segment limit field contains the length of the segment.

- Fig. Q.17.3 shows an address translation in segmentation.

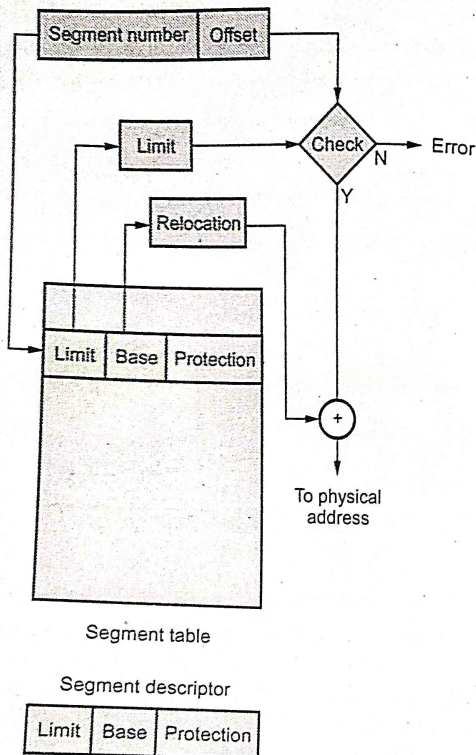


Fig. Q.17.3 Address translation in segmentation

- Segment table contains the physical address of the start of the segment. Then add the offset to the base and generate the physical address.
- If the required reference is not found in one of the segment registers, then error is generated. At the same time, operating system does lookup in segment table and loads new segment descriptor into the register.

Advantages

1. It provides virtual memory.
2. Allows dynamic segment growth.
3. Segmentation assists dynamic linking.
4. Segmentation is visible.

Q.18 Write difference between segmentation and paging.

Ans. : Difference between Segmentation and Paging

Sr. No.	Segmentation	Paging
1.	Program is divided into variable size segments.	Program is divided into fixed size pages.
2.	User (or compiler) is responsible for dividing the program into segments.	Division into pages is performed by the operating system.
3.	Segmentation is slower than paging.	Paging is faster than segmentation.
4.	Segmentation is visible to the user.	Paging is invisible to the user.
5.	Segmentation eliminates internal fragmentation.	Paging suffers from internal fragmentation.
6.	Segmentation suffers from external fragmentation.	There is no external fragmentation.
7.	Processor uses page number, offset to calculate absolute address.	Processor uses segment number, offset to calculate absolute address.
8.	OS maintain a list of free holes in main memory.	OS must maintain a free frame list.

Q.19 Write short note on : Segmentation with paging.

- Ans. : • Most of the architecture support paging and segmentation. All the pages of segment need not be in main memory.
- It simplifies the memory allocation and speed increases. It requires a high speed register to store the base address of the segment map table.
 - For each segment, page table is created by OS. A pointer to the page table is kept in the segment's entry in the segment table.

- Segments are typically larger than pages.
- The base address get from the segment descriptor table is concatenated with the offset. This new address is referred to as a **linear address**. Linear address is generated by paging hardware.

Advantages

Combines all advantages of paging and segmentation.

Disadvantages

1. It increases hardware cost.
2. It increases processor overheads.
3. Dangers of thrashing.

6.6 : Placement Strategies

Q.20 Explain in brief the memory allocation algorithms with examples.

Ans. : Best fit : The allocator places a process in the smallest block of unallocated memory in which it will fit. For example, suppose a process requests 12 kB of memory and the memory manager currently has a list of unallocated blocks of 6 kB, 14 kB, 19 kB, 11 kB and 13 kB blocks. The best-fit strategy will allocate 12 kB of the 13 kB block to the process. Best fit chooses the block that is closest in the size to the request.

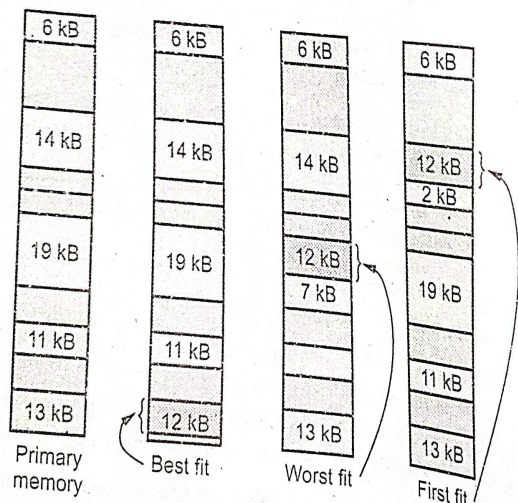


Fig. Q.20.1

Worst fit : The memory manager places a process in the largest block of unallocated memory available. The idea is that this placement will create the largest hold after the allocations, thus increasing the possibility that compared to best fit ; another process can use the remaining space. Using the same example as above, worst fit will allocate 12 kB of the 19 kB block to the process, leaving a 7 kB block for future use.

First fit : There may be many holes in the memory, so the operating system, to reduce the amount of time it spends analyzing the available spaces, begins at the start of primary memory and allocates memory from the first hole it encounters large enough to satisfy the request. Using the same example as above, first fit will allocate 12 kB of the 14 kB block to the process.

Q.21 Given a memory partitions of 100 K, 500 K, 200 K, 300 K and 600 K (in order) how would each of the first-fit, best-fit and worst-fit algorithms place processes of 212 K, 417 K, 112 K and 426 K (in order) ? Which algorithm makes the most efficient use of memory ?

[SPPU : May-19, End Sem, Marks 6]

Ans. : First-fit :

212 K is put in 500 K partition

417 K is put in 600 K partition

112 K is put in 288 K partition (new partition 288 K = 500 K - 212 K)

426 K must wait.

Best-fit :

212 K is put in 300 K partition

417 K is put in 500 K partition

112 K is put in 200 K partition

426 K is put in 600 K partition

Worst-fit :

212 K is put in 600 K partition

417 K is put in 500 K partition

112 K is put in 388 K partition (600 K - 212 K)

426 K must wait

In this example, Best-fit turns out to be the best.

6.7 : Virtual Memory

Q.22 What is virtual memory ? Explain demand paging with example.

[SPPU : Dec.-18, End Sem, Marks 6]

- Ans. : • Virtual memory is a method of using hard disk space to provide extra memory. It simulates additional main memory.
- Virtual memory is stored in the secondary storage device. It helps to extend additional memory capacity and work with primary memory to load applications.
 - The virtual memory will reduce the cost of expanding the capacity of physical memory. The implementations of virtual memory will differ for operating systems to operating system.
 - The logical addressable space is referred to as virtual memory. The virtual address space is much larger than the physical primary memory in a computer system. The virtual memory works with the help of secondary storage device and its speed is low compared to the physical storage location.
 - Virtual memory uses two types of addresses : Virtual address and physical address.
 - Virtual address is referred by process and physical address is actual address of the main memory.

Demand paging :

- In demand paging, a page is loaded into primary memory only when the process references it. It is the simplest fetch policy implemented in virtual memory. Fig. Q.22.1 shows demand paging.
- Demand paging guarantees that the system brings into primary memory only those pages that processes actually need. It involves interaction between hardware and software component of the virtual memory.

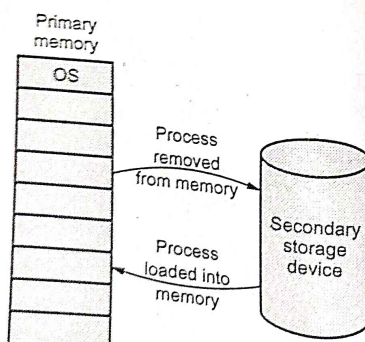


Fig. Q.22.1 Demand paging

- Paging with swapping method will become as demand paging concept. Process's logical address space is stored on the secondary storage device.
- Demand paging requires that the page map table for each process keep track of each page as it is loaded or removed from primary memory.
- When user want to execute a process, user swap a process into a memory. Lazy swapper is used for swapping the process into memory. Lazy swapper swaps the pages whenever needed

Q.23 What is structure of typical page table entry? What is significance of modified bit, reference bits, protection bits and present/absent bit in page table entry?

Ans. : • Fig. Q.23.1 shows page table entry.

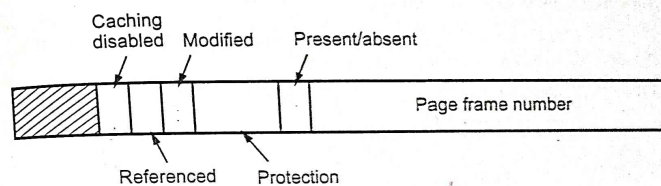


Fig. Q.23.1 Page table entry

- Page frame number : Most important field, the goal of the page mapping;
- Present /absent : If this bit is 1, the entry is valid and can be used;
- Protection bit is used for provide protection to the memory. Page table adds one more column for providing protection bit.
- Modified : Useable in reclaiming a page frame. If dirty, write back to disk.
- Referenced : The bit is set whenever a page is referenced. Help the OS choose a page to evict when a page fault occurs.
- Caching disabled : Useful for pages that map onto device register rather than memory

Q.24 Describe in detail the actions taken by the operating system when a page fault occurs.

Ans. : Steps in handling a page fault

- When page fault occurs, the system performs following steps :
- 1. Check the process control block table for reference which bit is set i.e. valid bit or invalid bit for memory access.
- 2. If invalid bit is set, user will terminate the process. If valid bit is set, then the page is loaded into memory.
- 3. Check for free frame for new page loading.
- 4. Start reading secondary storage device for desired page for allocated frame.
- 5. Required page is loaded into the memory after completion of read operation. PCB table is updated.
- 6. System restart the instruction that was interrupted by the error. Page is loaded in memory so process can use it.
- Fig. Q.24.1 shows page fault handling steps in demand paging.

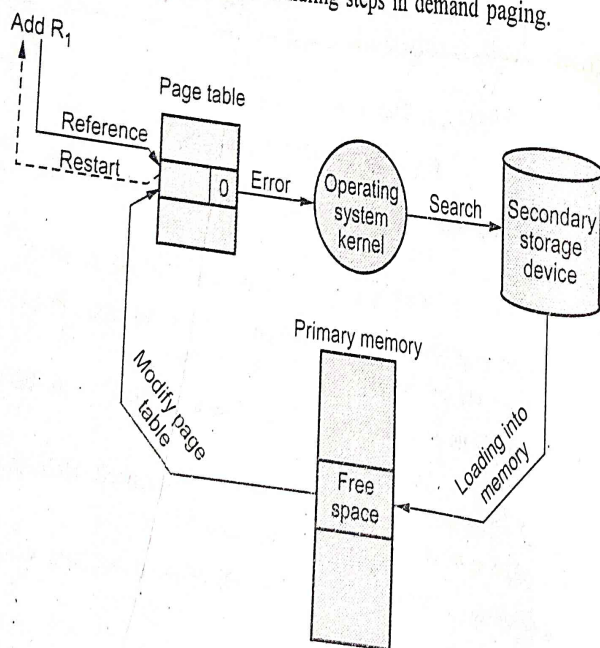


Fig. Q.24.1 Handling of page fault

6.8 : Swapping

Q.25 Write short note on swapping.

[SPPU : Dec-18,19, End Sem, Marks 5]

Ans. : Keeping all processes in memory all the time requires a large amount of main memory. For execution of the process, it must be swapped.

- **Swapping** is method of temporarily removing inactive process/programs from the physical memory.
- Inactive program means, which is neither executing on the CPU, nor performing an I/O operation.
- Fig. Q.25.1 shows swapping.

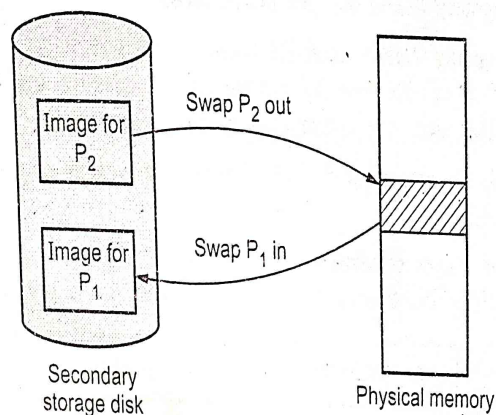


Fig. Q.25.1 Swapping

- When process P₂ send requests for an I/O operation, OS blocked P₂ process and will not return to the ready state for long period of time. Process P₂ is in blocked state. Process manager place the process P₂ in blocked state and inform to the memory manager regarding this action.
- Process manager moves a blocked process P₁ into the ready state and inform to memory manager. Process P₁ is loaded into main memory by memory manager when there is space available.
- Swapped out process is stored on the secondary storage disk. It contains executable image with code, data and stack.
- Relocation hardware supports for swapping. Because of address binding problem, swapping is difficult without relocation hardware.

- Round robin and priority based CPU scheduling algorithm uses the swapping concept.
- Time sharing system uses swapping. In time sharing, kernel can estimate when a program is likely to be scheduled next.
- A swapped out process will occupy same memory location when process swap in by OS. But this totally depends on address binding. If address binding is load time or assembly time, then swapped out process uses same memory location when swap-in. But execution-time binding uses different memory location.
- Secondary storage is required to implement the swapping technique.
- Content switching time increases when swapping increases. Major part of the swap time is transfer time. Swapping increases the operating system overhead due to the disk I/O involved.
- When swapping creates multiple holes in memory, it is possible to combine them all into one by moving all the processes to one side of memory. This technique is known as **memory compaction**.
- Compaction requires a lot of CPU time, so most operating systems not used this technique.
- Compaction means movement of processes in the memory during their execution. It is sometimes called dynamic relocation of a program.

6.9 : Page Replacement Policies

Q.26 Discuss page replacement algorithm.

- Ans. :**
- When a page fault occurs, page replacement algorithms are used for loading the page in memory and no free page frame exist in memory.
 - Page fault occurs if a running process references a nonresident page.
 - The goal of a replacement strategy is to minimize the fault rate. To evaluate a replacement algorithm, following parameters are used :
 1. The size of a page
 2. A set of reference strings
 3. The number of page frames.

- Given these inputs, we can determine the number of page faults and hence the page-fault rate. A reference string is a list of pages in order of their reference by the processor.
 - When we find that a page frame reference is in main memory then we have a page hit and when page fault occurs we say it is page miss. A poor choice of page replacement algorithm may result in lot of page misses.
 - Page replacement algorithm deals with how the kernel decides which page reclaim. Operating systems selects the local or global page replacement policy. Local replacement policy allocates a certain number of pages to each process. If a process needs new page, it must replace one of its own pages. If the OS uses a global policy, it can take a page from any process, using global selection criteria.
 - UNIX OS uses global page replacement policy.
- Q.27** List the page replacement algorithms. Explain LRU in detail.

[SPPU : May-16, End Sem, Marks 6]

Ans. : List of page replacement algorithms :

1. First-In-First-Out
2. LRU Page Replacement Algorithm
3. Optimal Page Replacement

The LRU page replacement policy :

- LRU stands for least recently used. This replacement policy chooses to replace the page which has not been referenced for the longest time. This policy assumes the recent past will approximate the immediate future. The operating system keeps track of when each page was referenced by recording the time of reference or by maintaining a stack of references.
- System must maintain a time stamp for every page that indicates the time of the last access. When deciding which page to remove, the operating system must scan all the pages of memory for finding the oldest time stamp. Instead of keeping an extra time stamp bits, LRU is implement by maintaining pages in an ordered list.
- System can manage LRU with a list called the LRU stack or the paging stack. In the LRU stack, the first entry describes the page referenced least recently, the last entry describes to the last page referenced. If a page is referenced, move it to the end of the list.
- LRU algorithm does not interfere with the locality of the process.

- Consider the reference string : 1, 2, 3, 4, 2, 5, 3, 4, 2, 6, 7, 8, 7, 9, 7, 8, 2, 5, 4, 9, 8, 2, 5, 4, 9
- Page frame size = 3

Reference string	1	2	3	4	2	5	3	4	2	6	7	8	7	9	7	8	2	5	4	9
Page frame 1	1	1	1	4	4	4	3	3	3	6	6	6	6	9	9	9	2	2	2	9
Page frame 2	-	2	2	2	2	2	2	4	4	4	7	7	7	7	7	7	7	5	5	5
Page frame 3	-	-	3	3	3	5	5	5	2	2	2	8	8	8	8	8	8	8	4	4
Page fault	PF	PF	PF	PF		PF	PF	PF	PF	PF	PF	PF	PF		PF		PF	PF	PF	PF

Total number of page fault = 16

- Page frame size = 4

Reference string	1	2	3	4	2	5	3	4	2	6	7	8	7	9	7	8	2	5	4	9
Page frame 1	1	1	1	1	1	5	5	5	5	6	6	6	6	6	6	6	2	2	2	2
Page frame 2	-	2	2	2	2	2	2	2	2	2	2	2	2	9	9	9	9	5	5	5
Page frame 3	-	-	3	3	3	3	3	3	3	3	7	7	7	7	7	7	7	7	4	4
Page frame 4	-	-	-	4	4	4	4	4	4	4	4	8	8	8	8	8	8	8	8	9
Page fault	PF	PF	PF	PF		PF				PF	PF	PF	PF		PF		PF	PF	PF	PF

Total number of page fault = 13

- LRU is a stack algorithm. Increase in memory size will never cause an increase in the number of page interrupts.

Q.28 Consider page sequence 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2 and discuss working of following page replacement policies. Also count page faults. (use no. of frames = 3).

- i) FIFO ii) LRU iii) Optimal [SPPU : May-18, End Sem, Marks 9]

Frame	2	3	2	1	5	2	4	5	3	2	5	2
0	2	2	2	2	5	5	5	5	3	3	3	3
1		3	3	3	3	2	2	2	2	2	5	5
2				1	1	1	4	4	4	4	4	2
Page fault	*	*		*	*	*	*		*		*	*

Number of page fault = 9.

- ii) LRU

Frame	2	3	2	1	5	2	4	5	3	2	5	2
0	2	2	2	2	2	2	2	2	3	3	3	3
1		3	3	3	5	5	5	5	5	5	5	5
2				1	1	1	4	4	4	2	2	2
Page fault	*	*		*	*		*		*	*		

Number of page fault = 7.

- iii) Optimal

Frame	2	3	2	1	5	2	4	5	3	2	5	2
0	2	2	2	2	2	2	4	4	4	2	2	2
1		3	3	3	3	3	3	3	3	3	3	3
2				1	5	5	5	5	5	5	5	5
Page fault	*	*		*	*		*		*	*		

Number of page fault = 6.

Q.29 Consider given page sequence a, b, c, d, c, a, d, b, e, b, a, b, c, d and the size of the frame is 4. Show the output of LRU and optimal policies, also count page faults.

[SPPU : Dec.-18, End Sem, Marks 6]

Ans. : LRU (Page frame = 4)

	a	b	c	d	c	a	d	b	e	b	a	b	c	d
1	a	a	a	a	a	a	a	a	a	a	a	a	a	a
2	-	b	b	b	b	b	b	b	b	b	b	b	b	b
3	-	-	c	c	c	c	c	c	e	e	e	e	e	d
4	-	-	-	d	d	d	d	d	d	d	d	d	c	c
Page fault	PF	PF	PF	PF					PF				PF	PF

Total number of page fault = 7

Optimal (Page frame = 4)

	a	b	c	d	c	a	d	b	e	b	a	b	c	d
1	a	a	a	a	a	a	a	a	a	a	a	a	a	d
2	-	b	b	b	b	b	b	b	b	b	b	b	b	b
3	-	-	c	c	c	c	c	c	c	c	c	c	c	c
4	-	-	-	d	d	d	d	d	e	e	e	e	e	e
Page fault	PF	PF	PF	PF										PF

Total number of page fault = 5

Q.30 Consider the following page reference string :

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 1, 5, 6

How many page faults would occur for the following replacement algorithms, assuming three frames.

- (1) LRU page replacement (2) FIFO page replacement
(3) Optimal page replacement

Ans. : (1) LRU page replacement :

	1	2	3	4	2	1	5	6	2	1	2	1	5	6
Page Frame 0	1	1	1	4	4	4	5	5	5	1	1	1	1	1
Page Frame 1	-	2	2	2	2	2	2	6	6	6	6	6	5	5
Page Frame 2	-	-	3	3	3	1	1	1	2	2	2	2	2	6
Page fault	PF	PF	PF	PF		PF	PF	PF	PF	PF			PF	PF

Total number of page fault = 11

(2) FIFO page replacement :

	1	2	3	4	2	1	5	6	2	1	2	1	5	6
Page Frame 0	1	1	1	4	4	4	4	6	6	6	6	6	5	5
Page Frame 1	-	2	2	2	2	1	1	1	2	2	2	2	2	6
Page Frame 2	-	-	3	3	3	3	5	5	5	1	1	1	1	1
Page fault	PF	PF	PF	PF		PF	PF	PF	PF	PF			PF	PF

Total number of page fault = 11

(3) Optimal page replacement

	1	2	3	4	2	1	5	6	2	1	2	1	5	6
Page Frame 0	1	1	1	1	1	1	1	1	1	1	1	1	5	5
Page Frame 1	-	2	2	2	2	2	2	2	2	2	2	2	2	2
Page Frame 2	-	-	3	4	4	4	5	6	6	6	6	6	6	6
Page fault	PF	PF	PF	PF			PF	PF					PF	

Total number of page fault = 7

Q.31 Explain with example Belady's anomaly.

Ans. : • Using FIFO algorithm for some reference strings actually generate more page faults when more page frames are allotted. This truly unexpected result was first demonstrated by Belady in 1970 and is known as Belady's anomaly.

- FIFO does not satisfy the inclusion property and is not a stack algorithm.
- A page replacement algorithm that satisfies the inclusion property is free from Belady anomaly.
- Consider the reference string : 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
- Page frame size = 3

Reference string	1	2	3	4	1	2	5	1	2	3	4	5
Page frame 1	1	1	1	4	4	4	5	5	5	5	5	5
Page frame 2	-	2	2	2	1	1	1	1	1	3	3	3
Page frame 3	-	-	3	3	3	2	2	2	2	2	4	4
Page fault	PF	PF	PF	PF	PF	PF	PF			PF	PF	

Total number of page fault = 9

Page frame size = 4

Reference string	1	2	3	4	1	2	5	1	2	3	4	5
Page frame 1	1	1	1	1	1	1	5	5	5	5	4	4
Page frame 2	-	2	2	2	2	2	2	1	1	1	1	5
Page frame 3	-	-	3	3	3	3	3	3	2	2	2	2
Page frame 4	-	-	-	4	4	4	4	4	4	3	3	3
Page fault	PF	PF	PF	PF			PF	PF	PF	PF	PF	PF

Total number of page fault = 10

- Paging algorithm has worse performance when the amount of primary memory allocated to the process is increased. The problem creates because the set of pages loaded with a memory allocation of 3 is not same as the with a memory allocation of 4.
- There are a set of demand paging algorithms whereby the set of pages loaded with an allocation of m frames is always a subset of the set of pages loaded with an allocation of $m + 1$ frame. This property is called the inclusion property.
- FIFO does not satisfy the inclusion property and is not a stack algorithm. LRU can be a stack algorithm.
- FIFO algorithm uses only the modified and status bits when swapping is used.

6.10 : Thrashing

[May-18, Dec-18, End Sem, Marks 4]

Q.32 What is thrashing ?

Ans. : • Thrashing occurs when a process does not have "enough" frames allocated to store the pages it uses repeatedly, the page fault rate will be very high.

- A process is thrashing if it is spending more time for paging in/out than executing. Since thrashing leads to low CPU utilization, the operating system's medium or long term scheduler may step in, detect this and increase the degree of multiprogramming.
- Local replacement algorithms can limit the effects of thrashing. If the degree of multiprogramming is increased over a limit, processor utilization falls down considerably because of thrashing. Fig. Q.32.1 shows thrashing.

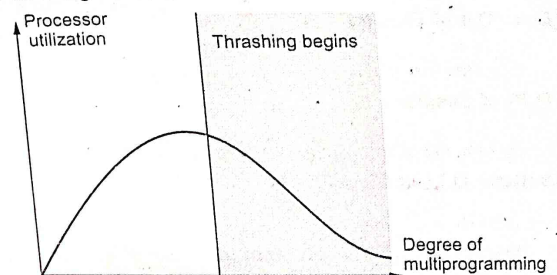


Fig. Q.32.1 Thrashing

- The selection of a replacement policy to implement virtual memory plays an important part in the elimination of the potential for thrashing. A policy based on the local mode will tend to limit the effect of thrashing. Replacement policy based on the global mode is more likely to cause thrashing. Since all pages of memory are available to all transactions, a memory-intensive transaction may occupy a large portion of memory, making other transactions susceptible to page faults and resulting in a system that thrashes.
- Thrashing is solved by using working set model and page fault frequency.

END...

SOLVED MODEL QUESTION PAPER (In Sem)
Systems Programming and Operating System
 T.E. (Computer) Semester - V [As Per 2019 Pattern]

Time : 1 Hour]

[Maximum Marks : 30]

N.B. : i) Attempt Q.1 or Q.2, Q.3 or Q.4.

ii) Neat diagrams must be drawn wherever necessary.

iii) Figures to the right side indicate full marks.

iv) Assume suitable data, if necessary.

Q.1 a) What is system software ? Explain any four system software's in brief. (Refer Q.4 of Chapter - 1) [4]

b) Compare one pass and two pass assembler. (Refer Q.29 of Chapter - 1) [3]

c) Explain the data structures required for TWO PASS assembler in details. (Refer Q.22 of Chapter - 1) OR [8]

Q.2 a) Define following system software components with suitable diagram, i) Compiler ii) Loader iii) Editor iv) Linker v) Debugger. (Refer Q.2 of Chapter - 1) [5]

b) Explain two pass translation of assembler. (Refer Q.19 of Chapter - 1) [4]

c) Explain in brief imperative statements, declaration statements and assembler directives with examples for assembly language programming. (Refer Q.13 of Chapter - 1) [6]

Q.3 a) Explain expansion time variable with example. (Refer Q.10 of Chapter - 2) [3]

SP&OS

M - 2

Solved Model Question Papers

b) What are the types of arguments used with macros ? Explain with suitable example. (Refer Q.8 of Chapter - 2) [4]

c) Explain design process task in macro expansion while designing two pass macro processor. (Refer Q.18 of Chapter - 2) [8]
 OR

Q.4 a) Define macro. What are the advantages of macro facility ? how are macros different from functions. (Refer Q.4 of Chapter - 2) [5]

b) What is interpreter ? Explain components of interpreter. (Refer Q.27 of Chapter - 2) [4]

c) What are the data structure used in design of macro processor ? (Refer Q.14 of Chapter - 2) [6]

SOLVED MODEL QUESTION PAPER (End Sem)
Systems Programming and Operating System
 T.E. (Computer) Semester - V [As Per 2019 Pattern]

Time : 2 $\frac{1}{2}$ Hours]

[Maximum Marks : 70]

N.B. : i) Attempt Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.

ii) Neat diagrams must be drawn wherever necessary.

iii) Figures to the right side indicate full marks.

iv) Assume suitable data, if necessary.

Q.1 a) What are types of loaders ? Explain compile and go loader scheme with advantages and disadvantages. (Refer Q.3 of Chapter - 3) [8]

b) Explain the formats of ESD, RLD, TXT and END cards with respect to direct linking loader with suitable example. (Refer Q.17 of Chapter - 3) [10]

OR

Q.2 a) Explain the working of direct linking loader by Pass2 Algorithm and Flowchart. (Refer Q.19 of Chapter - 3) [8]

b) Write short note on - Overlay structure (Refer Q.15 of Chapter - 3) [5]

c) Explain the concept of transfer vector in BSS loader. (Refer Q.10 of Chapter - 3) [5]

Q.3 a) Draw and explain process state transition diagram. (Refer Q.16 of Chapter - 4) [6]

b) Draw Gantt chart and calculate average turnaround time, average waiting time for the following processes using priority based (no-preemptive) scheduling and SJF (preemptive) scheduling policies. (Consider low no. as high priority) (Refer Q.32 of Chapter - 4) [6]

c) Explain CPU scheduling criteria. (Refer Q.30 of Chapter - 4) [5]

OR

Q.4 a) Draw Gantt chart and calculate average turnaround time, average waiting time for the following processes using SJF non preemptive and round robin with time quantum 2. (Refer Q.33 of Chapter - 4) [5]

b) Explain scheduling queue, ready queue and device queue with respect to process scheduling. (Refer Q.24 of Chapter - 4) [5]

c) Explain various multithreading model. (Refer Q.22 of Chapter - 4) [7]

Q.5 a) What is critical section? Give semaphore solution for producer - consumer problem. (Refer Q.19 of Chapter - 5) [8]

SP&OS

b) How livelock situation is avoided by using Dekker's solution? (Refer Q.5 of Chapter - 5) [6]

c) Explain monitor in brief. (Refer Q.13 of Chapter - 5) [4]

OR

Q.6 a) Consider the following snapshot of a system.

Processes	Allocation				Max.				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P ₀	2	0	0	1	4	2	1	2	3	3	2	1
P ₁	3	1	2	1	5	2	5	2				
P ₂	2	1	0	3	2	3	1	6				
P ₃	1	3	1	2	1	4	2	4				
P ₄	1	4	3	2	3	6	6	5				

Answer the following question using Banker's algorithm.

i) Illustrate that the system is in safe state by demonstrating an order in which the processes may complete?

ii) If a request from a process P₁ arrives for (1, 1, 0, 0), can the request be granted immediately?

iii) If the request from P₄ arrives for (0, 0, 2, 0), can the request be granted immediately? (Refer Q.28 of Chapter - 5) [8]

b) Write a semaphore solution for readers - writer's problem. (Refer Q.17 of Chapter - 5) [6]

c) Explain the following terms:

i) Critical section ii) Race condition (Refer Q.2 of Chapter - 5) [4]

Q.7 a) What is paging ? Explain address translation in paging.
(Refer Q.11 of Chapter - 6) [6]

b) Given a memory partitions of 100 K, 500 K, 200 K, 300 K and 600 K (in order) how would each of the first-fit, best-fit and worst-fit algorithms place processes of 212 K, 417 K, 112 K and 426 K (in order) ? Which algorithm makes the most efficient use of memory ?
(Refer Q.21 of Chapter - 6) [6]

c) How memory is protected by using relocation and limit register ? (Refer Q.8 of Chapter - 6) [5]

OR

Q.8 a) What is virtual memory ? Explain demand paging with example. (Refer Q.22 of Chapter - 6) [6]

b) Consider page sequence 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2 and discuss working of following page replacement policies. Also count page faults. (use no. of frames = 3).

i) FIFO ii) LRU iii) Optimal (Refer Q.28 of Chapter - 6) [8]

c) How the user program is processed ? Also explain binding of data and instruction to memory. (Refer Q.3 of Chapter - 6) [3]

END...